

MPCS 52553 — Software Quality Assurance

Assignment 5: Performance Testing

Name: Fatima Madey

Date: May 8, 2026

GitHub Repository: <https://github.com/fatimamadey/mpcs-sqa/tree/main/assignment5>

Objectives

By completing this assignment you will demonstrate the ability to:

- Distinguish functional regression testing from performance regression testing and select the appropriate approach for a given change.
 - Plan and execute a server-side performance test using an industry tool, defining operational and load profiles, SLA thresholds, and percentile-based metrics.
 - Evaluate client-side performance on both mobile and desktop.
 - Interpret performance test results and propose a concrete remediation.
-

Exercise 1: Functional vs Performance Regression Testing

Definitions

Functional Regression Testing is testing that verifies existing application features continue to behave correctly by producing the right outputs for given inputs—after a code change has been introduced. It either passes or fails. Functional regression suites would have things like unit, integration, and end-to-end tests that exercise business logic, data transformations, and UI workflows. The goal is to prevent regressions in correctness, not in speed. Common tools are pytest, JUnit, and Selenium/Playwright.

Performance (Non-Functional) Regression Testing is testing that verifies a system continues to meet quantitative speed, stability, and scalability benchmarks after a code change. It measures metrics like response time percentiles (p50, p95, p99), error rate under load, and throughput (requests per second). A test fails when a metric crosses a pre-defined SLA threshold—for example, if the p95 latency of an API endpoint rises from 80 ms to 520 ms after a refactor. Performance regression testing is conducted under realistic or stress-level load using tools such as k6, JMeter, or Locust, and results are compared against a baseline captured from the previous build.

TLDR Functional regression testing asks "does it work?" (correct output) while performance regression testing asks "does it perform?" (fast and stable under load). A system can pass all functional tests but still suffer a performance regression, like a refactored SQL query returning correct rows but 20x more slowly.

Scenarios

Scenario 1 — New validation rule added to the grade submission form to prevent missing fields

Functional Regression Testing

This change adds new business logic at the form-validation layer. It would need **functional** tests to verify things like submitting a form with a missing required field is rejected with the correct error message, that a fully populated form is accepted, and that existing valid-submission flows still work as expected. There is no new database query, no change to resource-intensive backend computation or infrastructure so a performance regression test would not give us anything meaningful.

Scenario 2 — Backend logic optimized to fetch assignment data more efficiently

Functional and Performance Regression Testing

The functional test is required to show that the optimized code is still returning the same assignment data—correct records, correct ordering, correct payloads. And the stated goal of the change is improved efficiency, so a **load test** should be run to verify the optimization actually improves (or at least does not degrade) performance under concurrent traffic. The metric to compare against the previous build is the **p95 response time** of the assignment-fetch endpoint (e.g., `GET /api/assignments`) at a realistic peak VU count; a secondary metric is throughput (RPS). If the optimized build does not improve p95 by a meaningful margin compared to the baseline, the optimization may not be production-ready.

Scenario 3 — Database index added to improve the speed of grade retrieval

Performance Regression Testing

An index addition does not alter the query's logical output—the same rows are returned in the same order—so there is no functional behavior change to verify. The correct regression test is a **load test** targeting the grade-retrieval endpoint (e.g., `GET /api/grades`), comparing the **p95 query response time** before and after the index is applied. If the index is effective, p95 should decrease; if p95 is unchanged or worse (e.g., index overhead on small tables), there needs to be an investigation. A stress test would also be helpful to validate that the index holds up under peak concurrency.

Scenario 4 — UI redesign changes the assignment submission button's location and styling

Functional Regression Testing

This change is purely visual so the appropriate tests are functional end-to-end tests (e.g., Playwright) that locate the submission button by its new position or accessible label and verify it still submits the form correctly, triggers the correct confirmation message, and does not break adjacent UI flows. There is no backend logic change and no new server-side computation, so a performance regression test would add no value. If the redesign introduces new third-party assets (fonts, analytics scripts), a client-side performance test (Lighthouse) could be run, but a server-side performance regression test is not needed.

Exercise 2: Server-Side Performance Testing

Tool: k6 v1.7.1

Target application: JSONPlaceholder public REST API (<https://jsonplaceholder.typicode.com>)

Feature under test: Browse flow — list posts, view a post, read its comments

JSONPlaceholder is a free, public fake REST API widely used for testing and prototyping. It requires no authentication and returns realistic JSON payloads, making it an ideal target for demonstrating performance test patterns without infrastructure dependencies.

Profiles

Operational Profile

The operational profile simulates a typical read-heavy browse session:

Step	HTTP Request	Rationale
1	<code>GET /posts</code>	Landing page — user browses the post listing (~27 KB JSON)
2	<code>GET /posts/:id</code>	User clicks into a post (VU id mod 100 spreads requests across 100 distinct records)
3	<code>GET /comments? postId=:id</code>	User reads comments for that post

Each step is followed by a 1-second think time (`sleep(1)`) to simulate realistic pacing between user actions. This yields 3 HTTP requests per iteration with a ~3-second iteration cycle.

Reasoning: The three-step sequence mirrors a real browse flow on any content-aggregation platform (analogous to Canvas: list assignments → view one → read feedback). Using `__VU % 100 + 1` as the post ID distributes requests across all 100 available records, preventing cache saturation on a single resource.

Load Profile

Load Test

```
Ramp  0 → 50 VUs over 30 s    (warm-up)
Hold   50 VUs for 1 min        (steady state – expected peak)
Ramp  50 → 0 VUs over 30 s    (cool-down)
Total: ~2 min
```

Reasoning: 50 concurrent users represents a realistic peak for a mid-sized course platform. The gradual ramp prevents a cold-start spike from distorting steady-state metrics. The 1-minute hold at 50 VUs produces ~900+ iterations — more than enough for statistically stable percentile measurements.

Spike Test

Baseline 10 VUs for 30 s	(normal traffic)
Instant spike to 200 VUs	(sudden surge – e.g., assignment deadline)
Hold 200 VUs for 1 min	(peak spike)
Instant drop to 10 VUs	(surge ends)
Hold 10 VUs for 30 s	(confirm recovery)
Total: ~2 min	

Reasoning: A 20× surge from 10 to 200 VUs represents a flash-crowd event (e.g., all students hitting the system at assignment deadline). The instant ramp (**duration: "0s"**) is intentionally aggressive to test whether the API can absorb sudden load without queuing or erroring. The recovery phase confirms that response times return to baseline after the spike.

SLAs (defined before running)

Metric	Threshold	Justification
p95 response time (http_req_duration)	< 500 ms	Standard web API SLA; aligns with Google's "good" LCP threshold
Error rate (http_req_failed)	< 1 %	Up to 1% HTTP failures is tolerable under load; above this, UX degrades noticeably
Throughput (http_reqs)	> 10 req/s	Minimum viable throughput at expected traffic levels

Test Setup (Script)

File: **scripts/loadtest.js** (load test) and **scripts/spiketest.js** (spike test)

Both scripts share the same operational profile. Below is the configuration from **loadtest.js**:

```
// scripts/loadtest.js
export const options = {
  summaryTrendStats: ['avg', 'min', 'med', 'max', 'p(90)', 'p(95)', 'p(99)'],
  stages: [
    { duration: "30s", target: 50 }, // ramp up
    { duration: "1m", target: 50 }, // steady state
    { duration: "30s", target: 0 }, // ramp down
  ],
  thresholds: {
    http_req_duration: ["p(95)<500"],
    http_req_failed: ["rate<0.01"],
    req_posts_list: ["p(95)<500"],
    req_post_detail: ["p(95)<500"],
    req_comments: ["p(95)<500"],
  }
}
```

```
    },  
  };
```

The spike test uses the same profile with stages **10 VUs** → **200 VUs (instant)** → **200 VUs** → **10 VUs (instant)** → **10 VUs**.

Screenshot location: [screenshots/loadtest_setup.png](#) — terminal showing the script file contents

Screenshot location: [screenshots/spiketest_setup.png](#) — terminal showing the spike script

Test Execution

Both tests were run locally on macOS (Apple M-series) against the live JSONPlaceholder API.

Commands:

```
# Load test  
k6 run scripts/loadtest.js  
  
# Spike test  
k6 run scripts/spiketest.js
```

Screenshot location: [screenshots/loadtest_run.png](#) — live terminal showing VU ramp and iteration counter

Screenshot location: [screenshots/spiketest_run.png](#) — live terminal showing VU jump from 9 to 200 at 1m01s

Test Results

Load Test Summary

Duration: 2m 02s | **Max VUs:** 50 | **Total iterations:** 1,497

■ THRESHOLDS

http_req_duration
✓ 'p(95)<500' p(95)=25.94ms

http_req_failed
✓ 'rate<0.01' rate=0.00%

req_comments
✓ 'p(95)<500' p(95)=25.31ms

req_post_detail

✓ 'p(95)<500' p(95)=22.59ms

req_posts_list

✓ 'p(95)<500' p(95)=26.71ms

■ TOTAL RESULTS

checks_total.....: 4491 36.78/s
checks_succeeded....: 100.00% 4491 out of 4491
checks_failed.....: 0.00% 0 out of 4491

✓ posts list 200
✓ post detail 200
✓ comments 200

CUSTOM

req_comments.....: avg=21.49ms med=22.29ms p(90)=23.95ms
p(95)=25.31ms p(99)=30.49ms max=87.76ms
req_post_detail....: avg=17.27ms med=17.06ms p(90)=21.56ms
p(95)=22.59ms p(99)=25.84ms max=69.18ms
req_posts_list.....: avg=22.66ms med=23.57ms p(90)=25.93ms
p(95)=26.71ms p(99)=35.31ms max=52.74ms

HTTP

http_req_duration.: avg=20.47ms med=21.48ms p(90)=24.78ms
p(95)=25.94ms p(99)=30.78ms max=87.76ms
http_req_failed....: 0.00% (0 out of 4491)
http_reqs.....: 4491 36.78 req/s

EXECUTION

iterations.....: 1497 12.26/s
vus.....: 2 min=2 max=50
vus_max.....: 50 min=50 max=50

Key metrics (load test):

Metric	Value	SLA	Status
p50 response time	21.48 ms	—	—
p95 response time	25.94 ms	< 500 ms	✓ PASS
p99 response time	30.78 ms	—	—
Error rate	0.00 %	< 1 %	✓ PASS
Throughput	36.78 req/s	> 10 req/s	✓ PASS

Screenshot: screenshots/loadtest_results.png

Spike Test Summary

Duration: 2m 03s | **Max VUs:** 200 | **Total iterations:** 4,147

■ THRESHOLDS

http_req_duration
✓ 'p(95)<500' p(95)=31.40ms

http_req_failed
✓ 'rate<0.01' rate=0.00%

spike_comments
✓ 'p(95)<500' p(95)=25.77ms

spike_post_detail
✓ 'p(95)<500' p(95)=24.05ms

spike_posts_list
✓ 'p(95)<500' p(95)=36.25ms

■ TOTAL RESULTS

checks_total.....: 12441 101.26/s
checks_succeeded....: 100.00% 12441 out of 12441
checks_failed.....: 0.00% 0 out of 12441

✓ posts list 200
✓ post detail 200
✓ comments 200

CUSTOM

spike_comments.....: avg=21.84ms med=22.46ms p(90)=24.57ms
p(95)=25.77ms p(99)=29.25ms max=77.71ms
spike_post_detail.: avg=19.49ms med=18.64ms p(90)=23.13ms
p(95)=24.05ms p(99)=33.67ms max=1.01s
spike_posts_list..: avg=26.60ms med=26.21ms p(90)=32.71ms
p(95)=36.25ms p(99)=42.69ms max=77.88ms

HTTP

http_req_duration.: avg=22.64ms med=22.40ms p(90)=28.61ms
p(95)=31.40ms p(99)=39.24ms max=1.01s
http_req_failed....: 0.00% (0 out of 12441)
http_reqs.....: 12441 101.26 req/s

EXECUTION

iterations.....: 4147 33.75/s
vus.....: 3 min=1 max=200
vus_max.....: 200 min=200 max=200

Key metrics (spike test):

Metric	Value	SLA	Status
p50 response time	22.40 ms	—	—

Metric	Value	SLA	Status
p95 response time	31.40 ms	< 500 ms	✅ PASS
p99 response time	39.24 ms	—	—
Error rate	0.00 %	< 1 %	✅ PASS
Throughput	101.26 req/s	> 10 req/s	✅ PASS

Screenshot: screenshots/spiketest_results.png

Discussion and Reflection

Both tests passed with flying colors, the load test p95 came in at 25.94 ms against a 500 ms SLA, which is not even close. JSONPlaceholder is basically a CDN-backed API with no real backend computation, so that makes sense. The throughput of 36.78 req/s at 50 VUs also cleared the 10 req/s floor easily.

In the spike test when VUs jumped from 10 to 200, the `GET /posts` endpoint's p95 climbed from 26.71 ms (load test) to 36.25 ms. That endpoint returns the largest payload (~27 KB, all 100 posts), so it makes sense it would feel the surge first. Still, 36 ms is nowhere near the SLA. The spike throughput hit 101 req/s, so the API was essentially serving as fast as 200 users could throw requests at it.

What actually surprised me was how instantly everything recovered once VUs dropped back to 10. I expected some tail of elevated latency — like a connection pool draining slowly — but the drop was basically immediate.

Exercise 3: Client-Side Performance Testing

Tool: Google Lighthouse v13 (via `npx lighthouse` CLI, headless Chrome)

Target page: <https://books.toscrape.com> — homepage (content-heavy product listing page)

Feature under test: Initial page load performance — how quickly the above-the-fold content becomes visible and interactive

Books to Scrape is a publicly accessible, content-rich static site with 20 product cards, images, and navigation — making it a representative target for Core Web Vitals analysis.

Test Setup

```
# Mobile (simulated Moto G Power, 4G throttling)
npx lighthouse https://books.toscrape.com \
  --output=json \
  --output-path=/tmp/lighthouse_mobile.json \
  --form-factor=mobile \
  --only-categories=performance \
  --chrome-flags="--headless --no-sandbox"
```



```
# Desktop (no throttling, 1350×940 viewport)
npx lighthouse https://books.toscrape.com \
  --output=json \
  --output-path=/tmp/lighthouse_desktop.json \
  --preset=desktop \
  --only-categories=performance \
  --chrome-flags="--headless --no-sandbox"
```

Screenshot location: [screenshots/lighthouse_setup.png](#) — terminal showing the lighthouse command invocations above

Results

Screenshot location: [screenshots/lighthouse_mobile_results.png](#) — Lighthouse JSON output / browser DevTools Lighthouse panel for mobile

Screenshot location: [screenshots/lighthouse_desktop_results.png](#) — Lighthouse JSON output / browser DevTools Lighthouse panel for desktop

Mobile Results (Moto G Power emulation, 4G)

Metric	Value	Score	Rating
Overall Performance Score	95 / 100	—	 Good
FCP (First Contentful Paint)	1.9 s	0.86	 Good
LCP (Largest Contentful Paint)	2.3 s	0.93	 Good
TBT (Total Blocking Time)	0 ms	1.00	 Good
CLS (Cumulative Layout Shift)	0	1.00	 Good
Speed Index	4.1 s	0.79	 Needs Improvement
Time to Interactive (TTI)	2.3 s	0.99	 Good
TTFB (Server Response Time)	40 ms	1.00	 Good

Desktop Results (no throttling, 1350×940)

Metric	Value	Score	Rating
Overall Performance Score	100 / 100	—	 Good
FCP (First Contentful Paint)	0.4 s	1.00	 Good
LCP (Largest Contentful Paint)	0.5 s	1.00	 Good
TBT (Total Blocking Time)	0 ms	1.00	 Good
CLS (Cumulative Layout Shift)	0.002	1.00	 Good
Speed Index	0.5 s	1.00	 Good

Metric	Value	Score	Rating
Time to Interactive (TTI)	0.5 s	1.00	 Good
TTFB (Server Response Time)	50 ms	1.00	 Good

Improvement Analysis

Identified area of improvement: Mobile Speed Index

Speed Index measures how quickly the above-the-fold content is *visually complete*. It captures the progressive paint of pixels over time, not just when the first or largest element appears. Mobile's Speed Index of 4.1 s (score 0.79) is the only metric that falls below "Good" across both platforms. Desktop's Speed Index was 0.5 s (perfect score), which is not a concern.

Root cause: Books to Scrape displays 20 product card images on the homepage. On mobile with 4G throttling, these images load sequentially after the first paint, delaying visual completeness even though the LCP element (the first visible image) resolves quickly at 2.3 s. The browser must paint many below-the-fold images before Lighthouse considers the viewport visually complete, and the network constraint on mobile makes this much slower than desktop.

Proposed remediation:

Lazy-load below-the-fold images. Add `loading="lazy"` to `<img>` tags for product cards that are not in the initial viewport. This defers off-screen image fetches, freeing up bandwidth for above-the-fold rendering and reducing the time to visual completeness on constrained networks.

Discussion and Reflection

Desktop got a perfect 100/100, which was surprisingly good. Every single metric was green, including a Speed Index of 0.5 s. Mobile came in at 95/100, which is still a solid score, but the Speed Index of 4.1 s is the clear weak spot. This makes intuitive sense: without CPU and network throttling, desktop can paint images fast enough that visual completeness is basically instant. On mobile with 4G simulation, the same 20 product images take noticeably longer to load, even though the page itself is interactive quickly.

The thing that I learned through this exercise was the gap between "interactive" and "visually complete." On mobile, TTI was 2.3 s — meaning you could tap things almost immediately — but Speed Index was 4.1 s, meaning you were staring at blank product card placeholders for a couple of seconds after that. That is a real UX problem: a user might think images failed to load. But you would never catch it from k6, because k6 only sees HTTP response times. The server returns 200s fast, but the browser still has to fetch and paint 20 images one by one over a throttled connection. That is exactly why you need both tools — k6 tells you if the backend is healthy, Lighthouse tells you what the user actually experiences.

References

- *k6 documentation*. <https://grafana.com/docs/k6/latest/>
- *Free fake REST API for testing and prototyping*. <https://jsonplaceholder.typicode.com>

- *Lighthouse documentation*. <https://developer.chrome.com/docs/lighthouse/>
- *Core Web Vitals*. <https://web.dev/vitals/>
- *Speed Index — Lighthouse performance scoring*.
<https://developer.chrome.com/docs/lighthouse/performance/speed-index/>
- *A fictional bookstore for web scraping practice*. <https://books.toscrape.com>
- *k6 load testing best practices*. <https://grafana.com/docs/k6/latest/using-k6/scenarios/>
- *Optimize Largest Contentful Paint*. <https://web.dev/optimize-lcp/>
- AI
 - Claude - Helped me write the operational profile in exercise 2 since I was a little confused on what to add. Also helped me organize data into tables and edit my final draft. Also helped me write the "Trend" code to track more specific metrics per step.
 - Copilot autocomplete for the bash script to run the tests and take screenshots, as well as the README.